

RAP 後端開發練習 7：Actions (自訂操作)

Lecture

這一課要解決什麼問題

rap02 ~ rap06 教的操作全部是標準 CRUD (**create**/**update**/**delete**) 加上框架自動觸發的 Determination / Validation——但真實業務常常需要一個**使用者主動觸發、有明確業務含義的操作**，例如「核准訂單」「標記完成」「重新計算」——這種不是單純改欄位值，而是代表一個業務事件的操作，RAP 用 **Action (自訂操作)** 表達。

延續 rap05 / rap06 的分工，但這一課有個好消息：

- **Part A：Managed Action 語法**——一樣能編譯、能啟用，一樣沒辦法真正執行 (Managed Runtime 白名單限制)
- **Part B：Unmanaged Action**——這是這系列課程第一次，**Unmanaged** 有真正的「宣告式」語法可以用，不用再手寫等效邏輯！查證官方文件確認 **Action** 沒有 rap05 / rap06 那種「非 **Draft** 不支援」的限制，Managed 跟 Unmanaged 都直接支援，已用 **programrun** 完整驗證：EML 呼叫 Action、資料真的被改、**RESULT** 真的正確回傳
- **Part C**：對照總表，順便總結 rap05 ~ rap07 這三課「宣告式機制在這系統的可執行性」全貌

Part A：Managed Action 語法 (知識儲備，這系統無法執行)

在看完整範例之前，先認識會用到的語法元素 (查證官方 ABAP 語言文件 **ABENBDL_ACTION_NONFACTORY** / **ABENBDL_ACTION1_ABEXA** 確認)：

- **action <名稱> result [1] \$self;**：寫在 BDEF 的 { } 區塊裡，宣告一個 Action。
 - **result [1] \$self**：宣告這個 Action 執行完後要回傳結果，**\$self** 代表「回傳型別跟這個實體自己一樣」(也可以指定成別的类型別)，**[1]** 是回傳筆數的基數 (Cardinality，這裡固定回傳 1 筆)——這是可選的，不需要回傳值的 Action 可以完全省略 **result** 子句。
 - Action 預設是 **instance-bound** (綁定在某個具體實例上執行，例如「核准『這張』訂單」)，如果要宣告成「跟整個實體集合有關、不綁定特定實例」的操作，要加 **static** 關鍵字 (這一課不用到)。
- ⚠ 跟 **Determination / Validation** 不同，**Action** 完全不需要 **obsolete** 語法——官方文件明講 Action 的自訂邏輯「must be implemented in the RAP handler method **FOR MODIFY**」，也就是說 Action 是 **create / update / delete** 同一個 Handler 類別 (**FOR MODIFY**)，只是用子句 **FOR ACTION <alias>~<action名稱>** 指定要綁定哪個 Action——這系統上 rap03 已經確認 **FOR MODIFY / FOR CREATE** 這個類別本身用官方現行語法就能編譯 (不像 Determination / Validation 有自己獨立的 **FOR DETERMINATION / FOR VALIDATION** 類別，那兩個類別的官方新語法才在這系統編譯失敗)，這次已實測驗證：**Action** 直接套用官方現行語法一次就編譯成功，完全不用套用 rap05 / rap06 學到的 **obsolete** 轉換規則。

延伸 rap05 / rap06 已經加過 Determination / Validation 的 **ZI_RAP02_TASK**，這是加上 Action 後的完整版本：

```
managed implementation in class zbp_i_rap02_task unique;
```

```

define behavior for ZI_RAP02_TASK alias Task
persistent table zrap02_task
etag created_at
lock master
{
    create;
    update;
    delete;

    field ( mandatory ) description;

    determination setCreationInfo on save { create; }
    validation validateStatus on save { field status; }

    action markDone result [1] $self;
}

```

markDone 這個 Action 要做的事：把 **status** 設成 **D** (Done) ——跟直接用 **update** 改 **status** 欄位效果類似，但 Action 表達的是「這是一個有業務含義的操作（標記完成），不是任意欄位修改」，也可以在 Action 裡加更多邏輯（例如同時記錄完成時間、檢查前置條件），這是 Action 跟單純 **update** 最大的差異。

Action 的 Handler Method：直接套用官方現行語法，不需要轉換

```

METHODS markDone FOR MODIFY
IMPORTING keys FOR ACTION Task~markDone RESULT result.

```

- **FOR MODIFY**：Action 的 Handler 類別跟 **create/update/delete** 共用（rap03 已教過），不是獨立類別。
- **FOR ACTION <alias>~<action名稱>**：**IMPORTING** 參數的 **FOR** 子句這次是 **ACTION**（不是 **CREATE/UPDATE/DELETE**），指定這個方法對應到 BDEF 宣告的哪個 Action。
- **RESULT result**：如果 BDEF 的 Action 有宣告 **result [...] ...**，Handler Method 就要對應宣告一個 **RESULT** 參數接收輸出——這裡不用 **TYPE**，型別一樣是編譯器從 BDEF 自動推導。

方法本體：

```

METHOD markDone.
MODIFY ENTITIES OF zi_rap02_task IN LOCAL MODE
ENTITY task
    UPDATE FIELDS ( status )
    WITH VALUE #( FOR ls_key IN keys (
        %key    = ls_key-%key
        status = 'D' ) ).

READ ENTITIES OF zi_rap02_task IN LOCAL MODE
ENTITY task
    FIELDS ( description status priority due_date created_at created_by )

```

```

    WITH CORRESPONDING #( keys )
    RESULT DATA(tasks).

result = VALUE #( FOR ls_task IN tasks (
    %key    = ls_task-%key
    %param = ls_task ) ).
ENDMETHOD.

```

- 邏輯很直觀：先用 **MODIFY ENTITIES ... IN LOCAL MODE** (rap05 教過的 Determination 專屬 EML 變體，這裡 Action 也用同一套) 把 **status** 改成 **D**，再 **READ ENTITIES** 把改完的完整資料讀回來。
- **result = VALUE #(... %key = ... %param = ls_task)**——**RESULT** 衍生型別的每一列有 **%key** (識別是哪一筆實例) 跟 **%param** (實際要回傳的資料內容，型別是實體本身)，跟前面 Determination / Validation 用的 **failed/reported (%key/%msg)** 是同一個「**%key** 開頭表識別、後面接內容」的命名慣例，換了場景但規律一致。
- **READ ENTITIES 要明確列出 FIELDS (...)**——這系統不支援官方教材偶爾出現的 **ALL FIELDS** 簡寫 (啟用時報 "FIELDS (" expected, not "ALL FIELDS".))，要逐一寫出欄位名稱，這系統目前遇到的所有 **READ ENTITIES** 語法都要遵守這條規則。

ZI_RAP02_TASK / ZBP_I_RAP02_TASK 加了 **markDone** 之後確認語法正確、成功啟用——一樣不要嘗試執行，原因同 rap05 / rap06 (Managed Runtime 白名單限制)。

Part B：Unmanaged Action——這次真的有宣告式語法可以用

查證官方文件 **ABENBDL_ACTION_NONFACTORY** 的「Consumption」(可用範圍) 章節，跟 rap05 / rap06 查過的 Determination / Validation 完全不同：

Actions are available for:

- Managed RAP BO
- Unmanaged RAP BO

沒有「**Caution: Not available for unmanaged, non-draft**」這種但書——Action 這個機制從設計上就同時支援 Managed 與 Unmanaged，不需要 Draft。這代表這一課的 Unmanaged 範例不用像 rap05 / rap06 那樣「自己在 CREATE 裡手寫等效邏輯」，可以直接在 **BDEF** 宣告 **action**，讓框架真正處理派發。

延伸 **ZI_RAP03_UMTEST**，加一個 **touch** Action (重新蓋 **created_at/created_by** 的時間戳，模擬「touch」一個檔案更新其時間戳的概念，呼應 rap05 的 Determination 主題，形成有趣的對照：rap05 是「建立時自動蓋」，這裡是「使用者主動要求重新蓋」)：

```

implementation unmanaged in class zbp_i_rap03_um4 unique;

define behavior for ZI_RAP03_UMTEST alias Test
lock master
{
    create;

    field ( readonly ) created_at, created_by;

```

```

    action touch result [1] $self;
}

```

Handler Method 宣告方式跟 Part A 完全一樣 (**FOR MODIFY + FOR ACTION**) · Unmanaged 唯一的差異是方法本體要自己寫 Open SQL (不像 Managed 用 **MODIFY ENTITIES IN LOCAL MODE** 委託框架) :

```

METHODS touch FOR MODIFY
    IMPORTING keys FOR ACTION test~touch RESULT result.

```

```

METHOD touch.
    DATA(ls_info) = determine_creation_info( ).

    LOOP AT keys INTO DATA(ls_key).
        UPDATE zrap03_umtest
            SET created_at = @ls_info-created_at,
                created_by = @ls_info-created_by
            WHERE id = @ls_key-id.
    ENDLOOP.

    LOOP AT keys INTO ls_key.
        SELECT SINGLE id, descr, created_at, created_by
            FROM zrap03_umtest WHERE id = @ls_key-id INTO @DATA(ls_data).
        IF sy-subrc = 0.
            APPEND VALUE #(
                %key    = ls_key-%key
                %param = ls_data ) TO result.
        ENDIF.
    ENDLOOP.
ENDMETHOD.

```

這裡直接重用 **rap05** 已經寫好的 **determine_creation_info()** 私有方法——這正是 rap05 提過的「這個私有方法故意取跟 Determination 一樣的名字，因為它扮演同樣的角色」這句話的具體體現：rap05 是在 **create** 方法裡呼叫它 (等效 Determination) · 這裡是在 Action 裡呼叫它 (一般的邏輯重用) ——同一段「算出建立資訊」的邏輯，可以被系統裡任何需要它的地方呼叫，這正是把邏輯拆成獨立方法的價值。

UPDATE ... SET ... WHERE id = @ls_key-id——注意 **SET** 子句多個欄位之間要用逗號分隔 (**created_at = ..., created_by = ...** · 不加逗號會報 **The elements in the "SET" list must be separated using commas.**) ; **WHERE** 子句不能明寫 **Client** 欄位 (**WHERE client = @sy-mandt AND id = ...** 會報 **The client field "CLIENT" cannot be specified in the WHERE condition. Client handling is performed by the compiler.**) ——這是新式 Open SQL (用 **@** escape 宿主變數) 的固定規則，編譯器自動處理 Client 比對，語法上不用 (也不能) 自己寫。

EML 呼叫 Action : **EXECUTE**

前面幾課用的都是 **CREATE/UPDATE** · 這一課第一次用到 EML 的 **EXECUTE** :

```

MODIFY ENTITIES OF zi_rap03_umtest
ENTITY Test
EXECUTE touch
FROM VALUE #( ( %key-id = 'ACT_TEST01' ) )
RESULT DATA(ls_result)
FAILED DATA(ls_failed_action)
REPORTED DATA(ls_reported_action).

```

- **EXECUTE** <action名稱> FROM VALUE #((%key-... = ...)) : 跟 CREATE/UPDATE 用 FIELDS (...) WITH VALUE #(...) 不同，EXECUTE 直接用 FROM VALUE #(...)——因為 Action 通常不需要「宣告要改哪些欄位」，只需要指定「對哪些實例執行」，所以每一列只要給 %key-<key欄位> 識別是哪一筆。
- ⚠️ **RESULT** 參數在 EML 呼叫端是直接的表格，不用像 FAILED/REPORTED 那樣加 -<alias> 分類——這是這一課另一個新發現，跟 rap06 學到的規則 (FAILED/REPORTED 在 FOR MODIFY 情境要用 -<alias>) 不完全一樣：RESULT (ls_result) 直接可以 LOOP AT ls_result，FAILED/REPORTED (ls_failed_action/ls_reported_action) 則要 ls_failed_action-test 才能存取——同一句 EML 陳述式裡，三個回應參數的型別規則並不統一，要逐一確認，不能假設三個都一樣。
- 讀取 **RESULT** 表格內容要透過 %param：LOOP AT ls_result INTO DATA(ls_res). ... ls_res-%param-descr ...——延續 Part A 已經教過的規則，RESULT 衍生型別的實際欄位資料都包在 %param 底下，不能直接 ls_res-descr。

✅ 驗證結果 (programrun 無頭執行，完全成功——這是真正的宣告式 Action，不是等效手寫版本)

```

before touch, created_at: 20260816135113.2901120
after EXECUTE touch, failed count: 0
after touch, created_at: 20260816135114.3324110
Action really re-stamped created_at: YES
result table lines: 1
result descr: Action Test

```

先建立一筆資料、記下當下的 created_at，等 1 秒後 EXECUTE touch，再次查詢——created_at 真的變了 (從 ...135113... 變成 ...135114...，差了約 1 秒，跟程式裡 WAIT UP TO 1 SECONDS 吻合)，RESULT 也正確回傳了改完之後的資料 (descr: Action Test)。跟 rap05 / rap06 不同，這裡驗證的是貨真價實透過 BDEF 宣告的 action + EML EXECUTE 觸發的邏輯，不是「因為框架不支援、只好自己在 CREATE 裡塞邏輯」的等效手寫版本。

Eclipse ADT：在既有 BDEF 裡加一個 Action——Step by Step

跟 rap05 / rap06 加 Determination / Validation 的操作模式一致：

1. Eclipse 打開目標 BDEF，在 { } 區塊裡手打 action <名稱> result [1] \$self; (或不需要回傳值就只寫 action <名稱>;)。
2. Managed BDEF 如果原本是單純 managed; (沒有實作類別)，要記得改成 managed implementation in class <類別名> unique; (跟 rap05 加 Determination 時一樣)；Unmanaged BDEF 本來就一定要 implementation unmanaged in class ...，不用另外改。

3. 存檔 (**Ctrl+S**) + 啟用 (**Activate**) 。
4. 在 Local Types Include 裡補上 **METHODS <名稱> FOR MODIFY IMPORTING keys FOR ACTION <alias>~<action名稱> [RESULT result].** 宣告 + 對應的 **METHOD ... ENDMETHOD.** 實作。如果這個 **BDEF** 是全新建立、還沒生成過 **Handler** 類別骨架，可以用 **rap03** 教過的 **Ctrl+1** (游標點在 **header** 類別名稱 → **Create behavior implementation class**) 一次生成整組骨架，新增的 Action 方法一樣要自己手動補 (Eclipse 目前不會針對「補一個新 Action」單獨產生快速修正) 。
5. 存檔 + 啟用整個類別。

練習：幫自己的 BDEF 加一個 Action

輪到你了，接續 **rap05 / rap06** 練習做的物件：

- ① 必做 (**Managed**，純語法練習)：對你 **rap05 / rap06** 練習用的 **Managed BDEF** (**ZI_RAP02_FLIGHT_PRACTICE**)，加一個 Action——例如「重新整理」某個欄位、或單純不帶邏輯只回傳 **\$self**。練習 **action ... result [1] \$self;** 語法 + **FOR MODIFY ... FOR ACTION ... Handler Method** 怎麼寫。驗收依據：**checkruns / sap_inactive_objects** 確認啟用成功即可。
- ② 選做 (**Unmanaged**，進階挑戰，真的能跑)：如果 **rap05 / rap06** 練習 ② 有自己建 **Unmanaged** 物件，幫它加一個真正的 Action (這次不是「等效手寫版本」，是真的宣告 **action** 讓框架派發)，用 **EML EXECUTE** 呼叫，驗證資料真的被改、**RESULT** 正確回傳。這題份量比前兩課的選做題輕 (不用像 **rap05 / rap06** 那樣繞路手寫等效邏輯，Action 本身就是宣告式的)，適合用來鞏固這一課的語法。

完成後跟我說一下建立過程跟最終狀態，我會幫你核對。

Part C：Managed vs Unmanaged Action 差異總表，以及 rap05 ~ rap07 三課回顧

	Managed	Unmanaged
宣告方式	BDEF 裡 action <名稱> [result [...]] ...; ，宣告式	同樣是 BDEF 裡 action ...; ，也是宣告式 (跟 Determination / Validation 不同，這裡 Unmanaged 沒有退化成手寫版本)
Handler Method 語法	FOR MODIFY ... FOR ACTION <alias>~<action> ，官方現行語法直接可用	完全相同的宣告語法
方法本體怎麼改資料	MODIFY ENTITIES ... IN LOCAL MODE (委託框架)	自己寫 Open SQL (UPDATE / INSERT / DELETE)，跟 create 方法一致的模式
這系統支援度	無限制 (跟 Draft 無關)	無限制 (官方文件沒有「非 Draft 不支援」這條，Determination / Validation 才有)
這系統能不能真正執行	✗ (Managed Runtime 白名單限制，跟 Action 語法本身無關)	✓ 已驗證成功 (真正的宣告式版本，不是手寫等效)

rap05 ~ rap07 三課的宣告式機制在這系統上的可執行性總覽：

機制	Managed 語法	Unmanaged 是否有宣告式語法	這系統上真正能執行的版本
Determination (rap05)	FOR DETERMINATION (obsolete)	✗ 官方明講不支援 (非 Draft)	只有 Unmanaged 手寫等效版本
Validation (rap06)	FOR VALIDATION (obsolete)	✗ 官方明講不支援 (非 Draft)	只有 Unmanaged 手寫等效版本
Action (rap07)	FOR ACTION (官方現行語法)	✓ 官方明講支援，無 Draft 限制	Managed / Unmanaged 都能寫，且 Unmanaged 真正能執行

這張表也解答了 rap05 一直沒有明講的問題：為什麼三個機制裡，只有 **Action** 的 **Handler Method** 不需要 **obsolete** 語法轉換？——因為 Determination / Validation 各自有專屬的 Handler 類別 (**FOR DETERMINE**/**FOR VALIDATE**，這系統只認得對應的 obsolete 版本 **FOR DETERMINATION**/**FOR VALIDATION**)，而 Action 沒有自己的專屬類別，是掛在 **FOR MODIFY** 底下 (跟 **create**/**update**/**delete** 共用)，**FOR MODIFY** 這個類別本身官方現行語法在這系統上從 rap03 就確認過可以直接編譯——這系統的「新舊語法落差」是精確發生在特定 **Handler** 類別層級，不是全面性的「這系統只認舊語法」，遇到新的 **RAP** 語言元素，要照 **rap05** 教的方法 (先試官方現行語法，編譯錯誤時再查 **obsolete** 版本) 逐一確認，不能一概而論。

學習目標

- 能寫出這系統適用的 Managed / Unmanaged Action 語法：`action <名稱> result [1] $self;`，知道 `result` 子句可省略
- 能寫出 Action 的 Handler Method 語法：`FOR MODIFY IMPORTING keys FOR ACTION <alias>~ <action名稱> RESULT result`，知道這是官方現行語法、不需要 **obsolete** 轉換
- 能講出 Action 跟 Determination / Validation 在「Unmanaged 是否有宣告式語法」上的關鍵差異，並說出背後原因 (Action 掛在 **FOR MODIFY**，Determination / Validation 各自有專屬 Handler 類別)
- 能用 `EML EXECUTE <action名稱> FROM VALUE #( ( %key-... = ... ) )` 呼叫一個 Action，知道跟 `CREATE/UPDATE` 用 `FIELDS ( ... ) WITH VALUE #( ... )` 語法不同
- 知道 **RESULT** 衍生型別的實際欄位資料要透過 `%param` 存取，且在 EML 呼叫端 **RESULT** 直接是表格、不用像 **FAILED/REPORTED** 加 `-<alias>`
- 能講出 rap05 ~ rap07 三課「Managed 語法可編譯但不能執行、Unmanaged 才是這系統真正能跑的版本」這條主線，以及 Action 是這條主線裡唯一的例外 (Unmanaged 也有真正的宣告式語法)

物件清單

物件	名稱	型別	可執行性
Managed Behavior Definition (延伸，加 action)	ZI_RAP02_TASK	BDEF/BDO	語法正確，CUD 無法執行
Managed 實作類別 (延伸，加 markDone)	ZBP_I_RAP02_TASK	CLAS/OC	語法正確，無法執行
Unmanaged Behavior Definition (延伸，加 action)	ZI_RAP03_UMTEST	BDEF/BDO	✓
Unmanaged 實作類別 (延伸，加 touch)	ZBP_I_RAP03_UM4	CLAS/OC	✓
Unmanaged EML 驗證程式	ZR_RAP07_ACTDEMO	PROG/P	✓ 已驗證成功

全部物件都在 \$TMP 套件，sap_inactive_objects 確認 0 筆殘留。

驗證方式

1. **Managed** 部分：checkruns / sap_inactive_objects 確認語法正確、成功啟用即可——不要嘗試執行，原因同 rap03 Part C / rap05 / rap06
2. **Unmanaged** 部分：已用 programrun 完整驗證成功，EXECUTE touch 後 created_at 真的被重新蓋上新的時間戳，RESULT 正確回傳改完的資料

思考題

1. 這一課的 markDone / touch 都是「不帶輸入參數」的 Action。如果想讓 Action 帶一個輸入參數（例如 touch 改成可以指定要不要順便更新 descr），BDEF 跟 Handler Method 的簽章大概要怎麼改？（提示：查官方文件 ABENBDL_ACTION_INPUT_PARAM，或回想這一課 result 子句的語法規則，輸入參數應該是類似的模式）
2. Part C 的表格整理出「Action 沒有 Draft 限制」的原因是它跟 create/update/delete 共用 FOR MODIFY 類別。那麼一個「跟 CRUD 標準操作綁得更緊」的自訂邏輯（例如這一課的情境），跟一個「完全獨立於任何實例、只是借用 RAP 框架傳輸資料」的操作，什麼情況下比較適合設計成 Action，什麼情況下該考慮宣告成 Function（這門課沒教，但官方文件裡跟 Action 平行的另一個機制）？
3. rap05 ~ rap07 都在同一個 ZI_RAP02_TASK / ZBP_I_RAP02_TASK 上疊加語法（Determination + Validation + Action）。如果之後要做 rap08 的 Header-Item 兩層架構，這種「在單一實體上不斷疊加各種機制」的作法，跟「兩層式資料模型」比起來，複雜度管理上有什麼本質差異？

答案

Managed：zi_rap02_task.bdef.abap（延伸版，含 Determination + Validation + Action）、zbp_i_rap02_task.clas.locals_imp.abap（延伸版，含三個 Handler Method）。**Unmanaged**：zi_rap03_umtest.bdef.abap（延伸版，加 action）、zbp_i_rap03_um4.clas.locals_imp.abap（延伸版，加 touch 方法）、zr_rap07_actdemo.prog.abap（EML 驗證程式，已驗證執行成功）。